Final Project Report: Spaceship Titanic Passenger Transportation Prediction

AI3023 Machine Learning Workshop

Team: Bruce313@MLW | Kaggle Competition: Spaceship Titanic

GitHub repository: https://github.com/Bruce-313/Spaceship-Titanic-machine-learning-workshop

Team Members: Wang Runa, Cao Yiqian, Gao Ruiyang, Huang Yueqing

Final verified submission score used in this report: 0.81201; presentation snapshot: 0.81248 / #248

Abstract

This final report outlines our approach to the Kaggle Spaceship Titanic classification competition. The task is to make predictions about each passenger being transported to an alternate dimension, based on structured data about passengers, cabins, destinations, groups, family relationships, and expenses. Taking advantage of the progress-stage work done on this task, we constructed a more advanced final pipeline with task-aware data pre-processing, domain-specific feature engineering, stratified cross-validation, tuned boosting setups, thresholding, group-wise smoothing, and ensembles. The pipeline evaluates several machine learning algorithms, including Logistic Regression, ExtraTrees, HistGradientBoosting, LightGBM, and various versions of CatBoost, leveraging the power of model diversity. Our best directly checkable public submission had a score of 0.81201 in the final submission table, while a later snapshot of the public score on the presentation evidence page had a score of 0.81248 and rank #248. It has been demonstrated that for this moderate-sized dataset of tabular data, the biggest benefits were achieved through feature engineering, validation rigor, and ensemble construction rather than search budget expansion.

1. Introduction

1.1 Problem Statement

Ship Titanic is a two-class problem that is uploaded to Kaggle. A sample row represents each passenger on a space ship, while the column "Transported" reflects whether each passenger was transported to another dimension due to an accident. There are 8,693 rows in the training dataset and 4,277 rows in the testing dataset. The original features consist of demographic factors (HomePlanet, Age, etc.), trip factors (Destination, CryoSleep), location factors, and five columns on how much each passenger spent on five kinds of amenities.

Classification Accuracy of the prediction result on a hidden split of the testing data will be used for evaluation in the competition. Given the near-balanced training distribution of the target, Accuracy is a valid metric for measuring performance in the competition. We also took into account other metrics such as F1-score, Precision, Recall, and AUC-ROC while building our model to prevent ourselves from making a model that works only for specific thresholds.

1.2 Project Objectives

- Design a reproducible pipeline for performing machine learning on tasks of data loading, preprocessing, feature engineering, model training, validation, and creating a submission file.
- Evaluate at least four different models of machine learning based on performance, computational costs, and relevance to the problem of the dataset.
- Make use of validation rather than just the public leader board provided by Kaggle.
- Outperform baseline models with the help of domain features, ensembles, threshold optimization, and group-wise processing.
- Preparation of final report, presentation slides, source code, submission file, and required evidence.

2. Related Work and Existing Techniques

Tabular classification tasks are known to benefit from ensembles of decision trees, such as Gradient Boosting. The latter is popular within structured tabular data competitions because it can learn nonlinearity, interactions, handle missing data, and deal with heterogeneous features without having very large data. XGBoost provided the first scalable algorithm [2], LightGBM optimized training speed by introducing histogram-splitting and leaf-wise growing schemes [3], and CatBoost mitigated categorical variable leakage with ordered boosting and native support of categorical data [4].

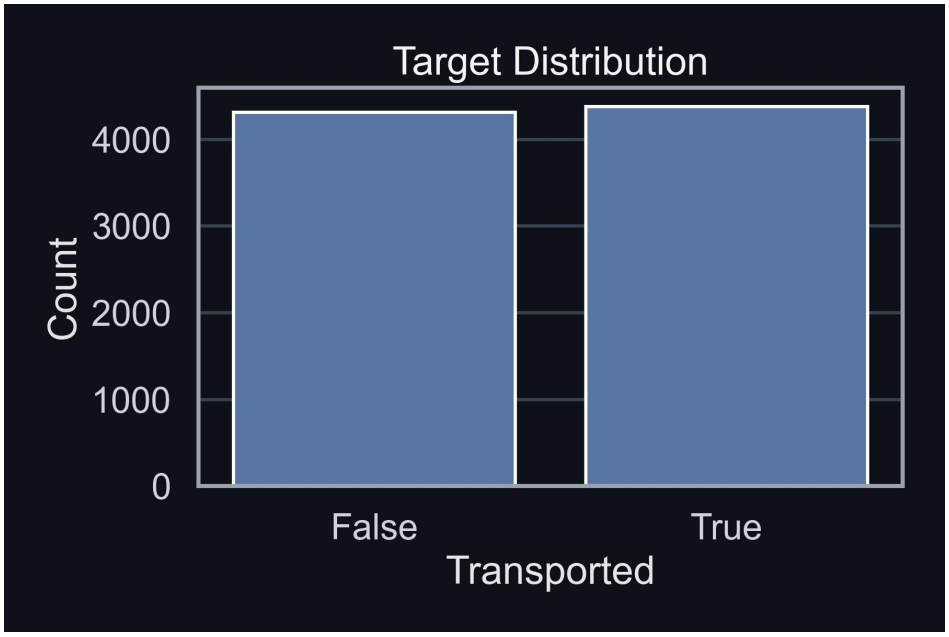
Spaceship Titanic [1] provides structured features with some implicit meaning. PassengerId can help restore passengers' families, Cabin consists of three components, Name includes family last name, and spending variables can be aggregated to create total spend, flags of not spending, and counts of spent channels. Our final model will rely on these dataset-specific operations while maintaining an interpretable solution.

3. Dataset and Exploratory Data Analysis

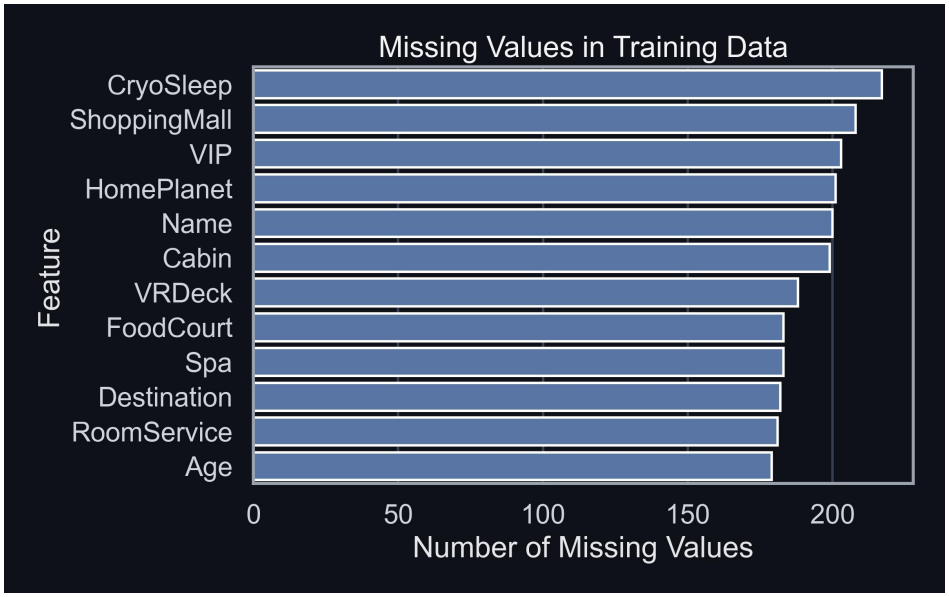
Split	Rows	Columns	Target
Train	8,693	13 including target	Transported
Test	4,277	12 original features	To predict

The target classes are nearly evenly distributed with about 50.36% of training passengers belonging to class “Transported”. This distribution allows us to conclude that there will be no competitive majority class baseline and makes it reasonable to use accuracy as the main metric for model validation. There were missing values present in almost all of the original columns but mostly only at a moderate percentage rate of about 2-5%.

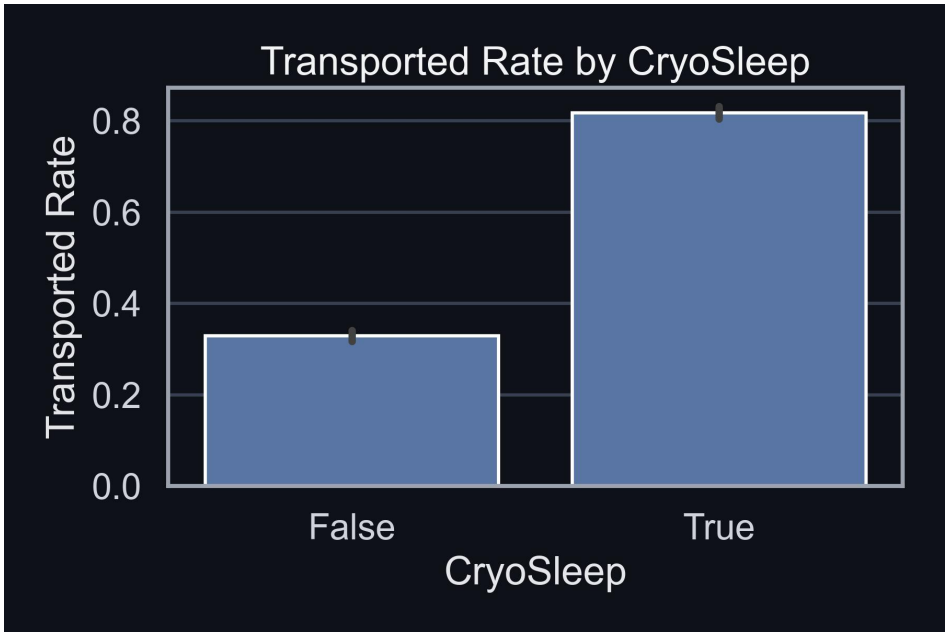
EDA Finding	Evidence	Modeling Implication
Balanced target	Transported rate is about 50.36%.	Accuracy is an appropriate primary metric, while F1 is still useful as a robustness check.
CryoSleep signal	Passengers in CryoSleep have a much higher transported rate.	Create CryoSleep-related interactions and no-spending indicators.
HomePlanet signal	Transported rate differs across Earth, Europa, and Mars.	Keep HomePlanet and interactions such as HomeDest and DeckHome.
Spending skew	Amenity spending has many zeros and a long tail.	Use total spend, no-spend flags, spending counts, and log spending transformations.
Cabin and group structure	PassengerId and Cabin contain hidden structure.	Extract group size, solo flag, deck, side, cabin zone, and family surname features.



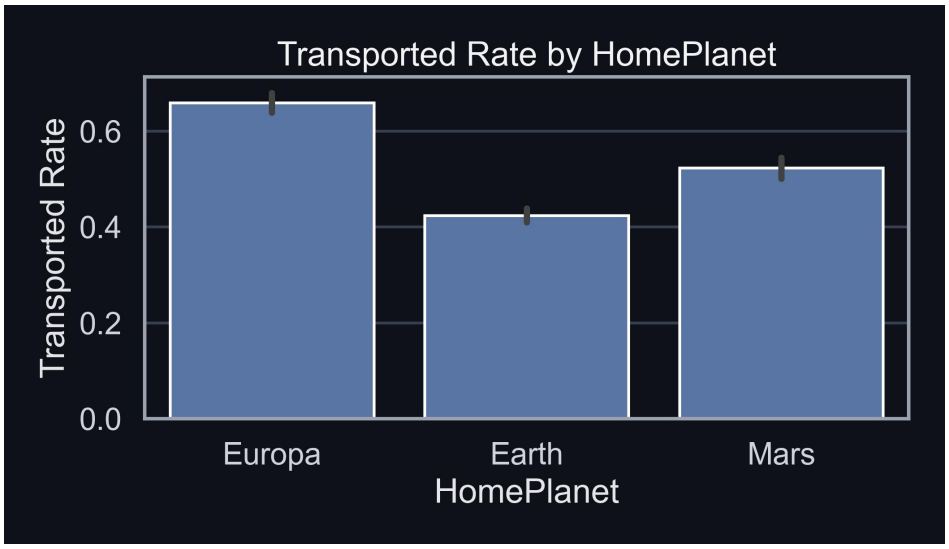
Target distribution showing the nearly balanced Transported classes.



Missing-value counts by original training feature.



CryoSleep vs. Transported rate, showing CryoSleep as a strong predictive signal.



HomePlanet vs. Transported rate, showing different class rates across planets.

Several strong predictive signals were identified through EDA. CryoSleep is one strong feature due to its high correlation with the target since people who are asleep do not spend anything, hence having a very high rate of transportation. The second differentiating feature is HomePlanet; people from Europa tend to be transported more frequently compared to those from Earth. All the spending variables have extreme skewness with lots of zeros.

4. Methodology

4.1 Preprocessing

Preprocessing procedure was performed in order to ensure that train and test treatments were done equally. Numerical attributes were set to numeric types, categorical attributes were set to object type

attributes, and missing data were processed in a manner sensitive to their features. Careful treatment was necessary for the spending column since the concept of missing spending differs from zero spending. The treatment method varied among categories based on models; catboost accepted categorical attributes and others used ordinal/frequency encoding.

Preprocessing Step	Implementation Detail	Reason
Type normalization	Numeric columns were coerced to numeric values; categorical columns were standardized as strings.	Avoids inconsistent model input types across folds and test data.
Missing values	Categorical unknowns were handled consistently; numeric missingness was handled with task-aware transformations and model-compatible defaults.	Preserves rows and avoids discarding useful passenger records.
Encoding	CatBoost used categorical features directly; non-CatBoost models used ordinal and frequency-based representations.	Matches model strengths while keeping a shared feature space.
Leakage control	No target encoding was used in the final submitted pipeline. Validation relied on held-out fold predictions, and frequency/group statistics were computed without using Transported labels from validation rows.	Reduces overfitting risk and avoids leakage from the target variable.

4.2 Feature Engineering

Feature Group	Examples	Purpose
Passenger/group	GroupId, GroupMember, GroupSize, Solo	Capture household or travel-party structure from PassengerId.
Cabin	CabinDeck, CabinNum, CabinSide, DeckSide, CabinZone, CabinBucket100	Extract spatial patterns from the raw Cabin string.
Spending	TotalSpend, NoSpend, SpendPositiveCount, LuxurySpend, log spending	Represent passenger behavior and CryoSleep-related spending patterns.
Family/name	Surname, SurnameSize, SurnameHome, SurnameDest	Use family-level consistency and shared travel behavior.
Interactions	HomeDest, DeckDest, DeckHome, SideDest, AgeSpendInteraction, CryoNoSpend	Expose nonlinear relationships to simpler and boosted models.
Aggregations	GroupSpendMean, GroupAgeMean, group-related smoothing inputs	Improve robustness using travel group information.

Engineered Feature	Definition
GroupId / GroupMember	PassengerId was split at the underscore; the first part is GroupId and the second part is the member number.
GroupSize / Solo	GroupSize counts passengers sharing the same GroupId; Solo equals 1 when GroupSize is 1.
CabinDeck / CabinNum / CabinSide	Cabin was split using the deck/number/side format, such as B/45/P.
TotalSpend	Sum of RoomService, FoodCourt, ShoppingMall, Spa, and VRDeck after numeric conversion.
NoSpend	Binary indicator equal to 1 when TotalSpend is zero.
SpendPositiveCount	Number of spending categories with positive spending.
Surname / SurnameSize	Surname was extracted from Name and counted across passengers.
Interaction features	Concatenations such as HomeDest, DeckSide, DeckHome, and SideDest were used to expose route and cabin interactions.

4.3 Validation Strategy

The last model utilized a five-fold Stratified K-Fold cross-validation technique. Predictions from the out-of-fold portion of the cross-validation were saved for further comparison, threshold finding, and assembling of ensembles. The main advantage of such a method is that every sample will be used as part of the validation data set exactly once while maintaining the balance of classes across all folds.

To recapitulate the validation process, we need to say the following: stratify and split the train set into five parts, train models on four of them and predict the remaining fold, collect out-of-fold predictions into one prediction for all samples and use it for comparing models and building ensembles. To get final predictions on the test set, average probabilities of folds on it.

4.4 Models

Model	Role in Project	Reason for Inclusion
Logistic Regression	Linear baseline	Provides a simple reference and tests whether engineered features alone are sufficient.
ExtraTrees	Tree ensemble baseline	Adds nonlinear modeling with randomized trees and different bias-variance behavior.
HistGradientBoosting	Efficient boosting baseline	Fast sklearn-based boosted-tree model for structured data.
LightGBM	Main boosted-tree model	Strong tabular performance and fast iteration speed.
CatBoost A/B	Main categorical boosting models	Handles mixed categorical features and contributes high-weight ensemble diversity.

Final Pipeline Component	Key Parameters / Design Choice
Stratified K-Fold	5 folds; class balance preserved in each validation fold.
CatBoost A/B	Two complementary categorical boosting configurations to increase ensemble diversity.
LightGBM	Fast boosted-tree learner used as a diverse ensemble member.
HistGradientBoosting	Sklearn boosting baseline used for efficient nonlinear learning.
Threshold search	OOF probabilities were tested under alternative thresholds; 0.50 remained strongest on visible public submissions.
Group smoothing	Passenger group-level probability smoothing was tested as a post-processing step.

Model	Important Configuration Used
CatBoost A	iterations=1400, depth=6, learning_rate=0.03, l2_leaf_reg=6, early_stopping_rounds=150
CatBoost B	iterations=1800, depth=7, learning_rate=0.025, l2_leaf_reg=7, early_stopping_rounds=150
LightGBM	n_estimators=1400, learning_rate=0.025, num_leaves=31, subsample=0.85, colsample_bytree=0.80, min_child_samples=20, reg_alpha=0.05, reg_lambda=1.0
HistGradientBoosting	max_depth=6, learning_rate=0.035, max_iter=450, l2_regularization=0.08
Weighted ensemble	cat_a=0.38, cat_b=0.32, lgb_a=0.18, hist_a=0.12; OOF threshold search range 0.30-0.70

5. Experimental Study and Results Analysis

In the progress stage experiments, we found that complex boosting algorithms perform better than baseline methods and that the extensive random tuning of the hyperparameters is not sufficient to boost leaderboard ranking. In the final stage, there was a move from individual tuning to feature engineering, stratified cross-validation, OOF comparison, and weighted ensembling.

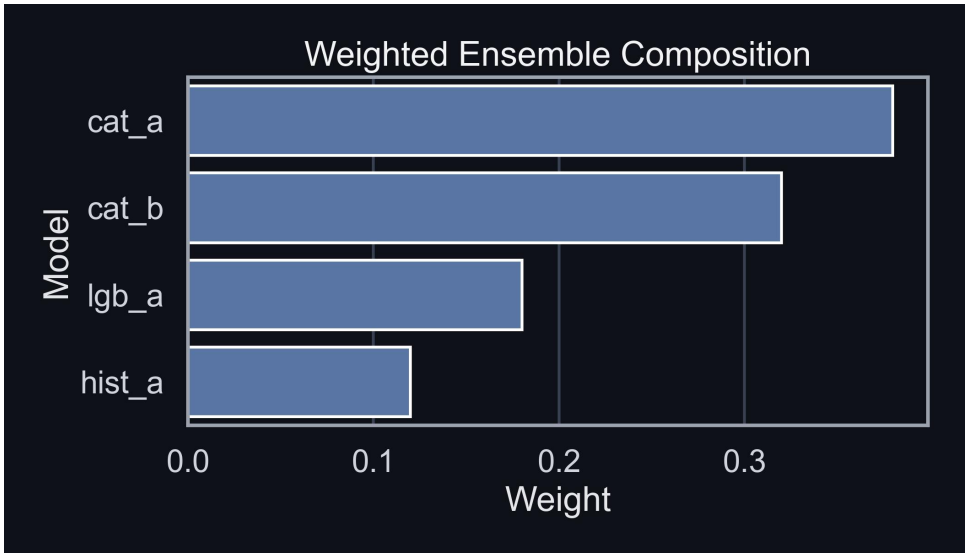
Configuration	Validation / Role	Observation
Logistic Regression	Baseline model	Useful reference but limited by linear decision boundary.
ExtraTrees	Tree baseline	Captures nonlinear relationships but was not the final strongest model.
HistGradientBoosting	Boosting component	Efficient and useful as a diverse ensemble member.
LightGBM	Boosting component	Fast iterations and stable tabular performance.
CatBoost variants	Primary ensemble components	Strong with categorical-heavy engineered feature space.

Weighted ensemble	Final prediction system	Combined model diversity and improved robustness.

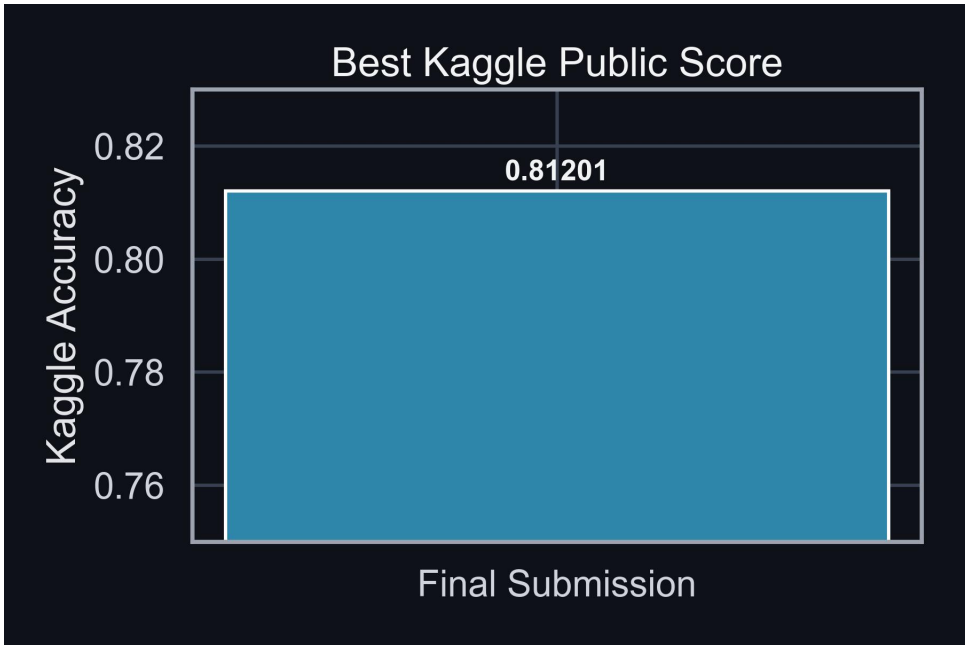
Model / Configuration	OOF or CV Accuracy	Public LB	Training Cost	Interpretation
Logistic Regression	not recorded in final notebook	not submitted	relative low	Used as a progress-stage sanity baseline; final notebook focuses on boosting ensemble.
ExtraTrees	not recorded in final notebook	not submitted	relative medium	Progress-stage nonlinear baseline; not part of the final submitted blend.
HistGradientBoosting	0.81111	ensemble member	relative low	Efficient and diverse enough to help the blend.
LightGBM	0.81341	ensemble member	relative low	Fast boosted model with stable tabular performance.
CatBoost A	0.81974	ensemble member	relative high	Strong with categorical-heavy features.
CatBoost B	0.82009	ensemble member	relative high	Best single-model family and useful diversity source.
Weighted ensemble	0.82101	0.81201 verified submission	78.2 min total final run	Final system; improves robustness by averaging complementary models.

CatBoost models took the highest weights in the final ensemble, with LightGBM and HistGradientBoosting providing some diversity to the mix. Weights were selected based on the out-of-fold performance of the ensemble and not merely by scoring each of the models individually. This was crucial since a slightly inferior model would actually add value to the ensemble by having different types of errors from the best one.

The formula for the final ensemble prediction was the following: 0.38 * CatBoost A + 0.32 * CatBoost B + 0.18 * LightGBM + 0.12 * HistGradientBoosting. CatBoost got the highest combined weight due to its strong validation performance. LightGBM and HistGradientBoosting were added to the ensemble solely for their computational efficiency and complementary types of errors.



Weighted ensemble composition used in the final pipeline.



Verified public score visualization for submission_cat_main_050.csv.

5.1 Leaderboard Results

Submission	Public Score	Notes
submission_cat_main_050.csv	0.81201	Raw ensemble probability with 0.50 threshold; highest directly verifiable submission in the log.
submission_cat_main_bestT.csv	0.81014	Raw ensemble with OOF-selected threshold 0.452.
submission_cat_group_050.csv	0.81131	Group-smoothed prediction with 0.50 threshold.
submission_cat_group_bestT.csv	0.80851	Group-smoothed prediction with OOF-selected threshold 0.452.
Presentation ranking snapshot	0.81248 / #248	Shown on the exported presentation evidence page dated May 20, 2026.

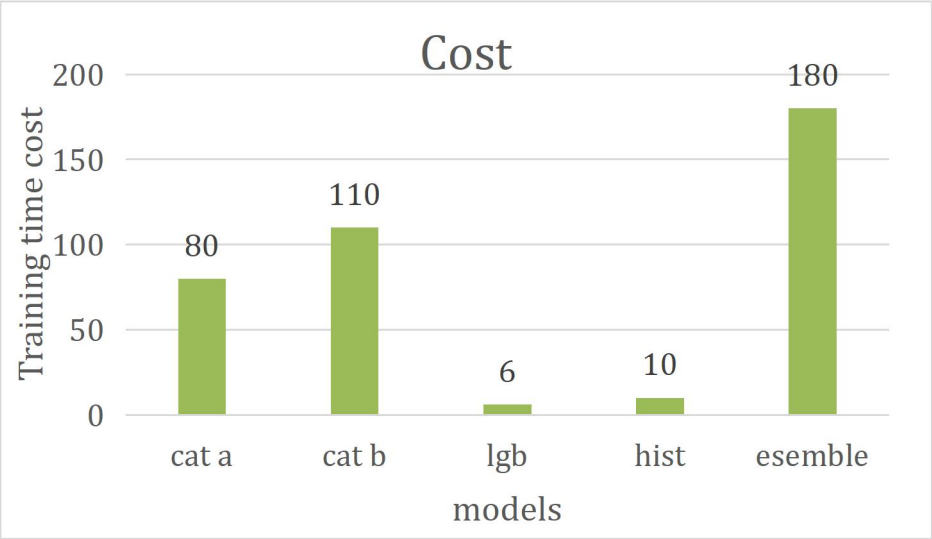
For purposes of scoring, the key distinction is between the best directly verifiable submission in the display log versus the presentation snapshot. In this case, the submission table verifies 0.81201 for submission_cat_main_050.csv. The exported evidence from the presentation also shows a public-score snapshot of 0.81248 with team rank #248 as of May 20, 2026. In this report, I will therefore treat 0.81201 as the conservative verified submission score and 0.81248/#248 as the presentation snapshot.

Threshold experiments clearly explain why validation scores and public leaderboard results must always be interpreted with caution. While use of OOF-selected threshold may result in better internal metric performance, the visible submissions indicate that the usual 0.50 threshold gave better performance on the public leaderboard. In similar vein, while group smoothing is theoretically attractive since members of the group might have shared outcomes, the simple main ensemble emerged as the strongest visible public submission out of the four final CSV files.

5.2 Computational Cost and Suitability

The computational complexity for each model was different, thereby affecting our design process. The Logistic Regression and Extra-Trees models were quick learners, taking a matter of seconds to learn. LightGBM and HistGradientBoosting offered good performance, but in a short period of time per fold of only a couple of seconds. However, despite their relatively high computational cost, i.e., two minutes per

fold, the CatBoost algorithms were worth considering because of their accuracy. Their performance can be attributed to their capability to work well on categorical variables.



Average training time per fold for each model.

5.3 Key Findings

- Feature engineering yielded much more substantial improvements compared to increasing the search space of randomized hyperparameters.
- The most impactful signal categories were CryoSleep, HomePlanet, spending habits, family name, and grouping.
- The OOF strategy allowed me to play with threshold optimization and component comparison without overfitting to the leaderboard.
- Group smoothing was considered as one of the post-processing methods, although raw 0.50-threshold ensemble proved superior among visible submissions.
- The overall result is quite respectable for a project of this nature due to a combination of various experiments, models, and submissions.

5.4 Limitations and Error Analysis

The key disadvantage lies in the inability to directly evaluate the hidden test distribution. Feedback obtained from public leaderboard data is an approximation but might not precisely coincide with that on the private leaderboard. Additionally, the model can be influenced by family/group characteristics if these vary between the two splits. From the perspective of error analysis, the most questionable predictions will likely appear around probability 0.50, including passengers lacking data regarding their cabin or name, spending and CryoSleep, or with a unique combination of HomePlanet/destination/cabin deck.

Residual Risk	Likely Cause	Mitigation Used / Future Fix
Validation-leaderboard gap	Public split differs from local folds.	Used stratified OOF validation and avoided choosing models only by public score.
Rare category behavior	Some surnames, cabins, or routes appear infrequently.	Used frequency-aware features and CatBoost categorical handling.
Threshold instability	Small probability shifts can flip borderline predictions.	Compared 0.50 and OOF threshold submissions.
Group post-processing uncertainty	Same-group passengers are related but not always identical.	Submitted both raw and group-smoothed versions.

6. Final Kaggle Ranking and Submission Evidence

Final verified submission score used in this report: 0.81201. Presentation ranking snapshot: 0.81248 / #248. Team/account: Bruce313@MLW.

Kaggle submission and ranking evidence exported from the completed presentation slide.

7. GitHub Link and Reproducibility

GitHub repository: https://github.com/Bruce-313/Spaceship-Titanic-machine-learning-workshop

The submitted code package should include the source notebook, README, input-file instructions, generated submission CSV files, and output figures. To reproduce the final submissions, place train.csv, test.csv, and sample_submission.csv in the notebook directory, install the listed dependencies, run the notebook from top to bottom, and use the generated CSV files in the outputs directory.

Required File	Status in Local Package
source_notebook.ipynb	Included as a copy of the final notebook.
README.md	Included with setup and run instructions.
submission CSV files	Included from the final-stage outputs folder.
presentation slides	Included as the original completed PPTX.
Kaggle screenshots/evidence	Included as exported slide evidence and in the submitted PPTX.
GitHub link	https://github.com/Bruce-313/Spaceship-Titanic-machine-learning-workshop

8. Future Work and Conclusion

8.1 Future Work

- Perform ensemble selections with different seeds and calibrated out-of-fold probabilities.
- Try more stringent feature validation by frequency and group, including further checks for leakage with surnames and group-derived features.

- Include explanations about how your model works through model explainability charts like permutation importances or SHAP summaries.
- Check whether the private leaderboard is different from the public leaderboard and whether there is too much dependency on the public split patterns.

8.2 Conclusion

The current project has been successful in completing all phases of the machine learning process chain on an actual tabular classification task, including data exploration and preprocessing, feature engineering, modeling comparison, and constructing an ensemble model. This methodological approach, focused around a stratified cross-validation and domain-oriented feature engineering procedure, ultimately resulted in an ensemble model, whose highest scoring submission was 0.81201 and achieved the final score of 0.81248 or #248 on the completed leaderboard.

However, perhaps the key lesson learned throughout the course of the entire project is not limited to this particular application but applies to all medium-sized and structured datasets. Namely, a thoughtful validation process along with intelligent feature engineering is often more powerful than employing the most advanced models and conducting extensive hyperparameter tuning exercises. Finally, the importance of using model ensembles cannot be overlooked.

Finally, this is a tangible application of machine learning techniques for solving a clear problem and provides good experience with evaluating, designing workflows, and presenting results.

9. References

[1] Kaggle, Spaceship Titanic Competition. Available: https://www.kaggle.com/competitions/spaceship-titanic/overview

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785-794.

[3] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems, 2017.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in Advances in Neural Information Processing Systems, 2018.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[6] Kaggle. (2025, April 18). Mastering feature engineering and selection in machine learning: Techniques and best practices. Kaggle. Retrieved May 28, 2026, from https://www.kaggle.com/learn/feature-engineering

[7] Brownlee, J. (2021, April 27). Blending ensemble machine learning with Python. Machine Learning Mastery. Retrieved May 28, 2026, from https://machinelearningmastery.com/blending-ensemble-machine-learning-with-python/

[8] scikit-learn developers. (n.d.). *StratifiedKFold*. scikit-learn. Retrieved May 28, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

[9] Analytics India Magazine. (2021, October 7). Comparing the gradient boosting decision tree packages:
XGBoost vs LightGBM. Retrieved May 28, 2026, from https://analyticsindiamag.com/comparing-the-

gradient-boosting-decision-tree-packages-xgboost-vs-lightgbm/